

■ 3.2.4 Integer and Number Theoretical Functions

<code>Mod[k, n]</code>	k modulo n (positive remainder from dividing k by n)
<code>Quotient[m, n]</code>	the quotient of m and n (integer part of m/n)
<code>GCD[n₁, n₂, ...]</code>	the greatest common divisor of $n_1, n_2, \dots$
<code>LCM[n₁, n₂, ...]</code>	the least common multiple of $n_1, n_2, \dots$

Some integer functions.

The remainder on dividing 17 by 3.

```
In[1]:= Mod[17, 3]
Out[1]= 2
```

The integer part of 17/3.

```
In[2]:= Quotient[17, 3]
Out[2]= 5
```

Mod also works with real numbers.

```
In[3]:= Mod[5.6, 1.2]
Out[3]= 0.8
```

Mod always gives a non-negative result.

```
In[4]:= Mod[-5.6, 1.2]
Out[4]= 0.4
```

For any integers a and b , it is always true that $b \cdot \text{Quotient}[a, b] + \text{Mod}[a, b]$ is equal to a .

The **greatest common divisor** function `GCD[n1, n2, ...]` gives the largest integer that divides all the n_i exactly. When you enter a ratio of two integers, *Mathematica* effectively uses `GCD` to cancel out common factors, and give a rational number in lowest terms.

The **least common multiple** function `LCM[n1, n2, ...]` gives the smallest integer that contains all the factors of each of the n_i .

The largest integer that divides both 24 and 15 is 3.

```
In[5]:= GCD[24, 15]
Out[5]= 3
```

<code>FactorInteger[n]</code>	a list of the prime factors of n , and their exponents
<code>Divisors[n]</code>	a list of the integers that divide n
<code>Prime[k]</code>	the k^{th} prime number
<code>PrimeQ[n]</code>	give <code>True</code> if n is a prime, and <code>False</code> otherwise

Integer factoring and related functions.

This gives the factors of 24 as $2^3, 3^1$. The first element in each list is the factor; the second is its exponent.

```
In[6]:= FactorInteger[24]
Out[6]= {{2, 3}, {3, 1}}
```

Here are the factors of a larger integer.

```
In[7]:= FactorInteger[111111111111111111]
Out[7]= {{3, 2}, {7, 1}, {11, 1}, {13, 1}, {19, 1},
         {37, 1}, {52579, 1}, {333667, 1}}
```

You should realize that according to current mathematical thinking, integer factoring is a fundamentally difficult computational problem. As a result, you can easily type in an integer that *Mathematica* will not be able to factor in anything short of an astronomical amount of time. So long as the integers you give are less than about 20 digits long, `FactorInteger` should have no trouble. Only in special cases, however, will it be able to deal with much longer integers. (You can make some factoring problems go faster by setting the option `FactorComplete->False`, so that `FactorInteger[n]` tries to pull out only one factor from n .)

Here is a rather special long integer.

```
In[8]:= 30!
Out[8]= 265252859812191058636308480000000
```

Mathematica can easily factor this special integer.

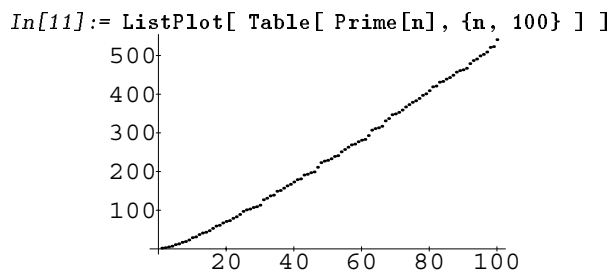
```
In[9]:= FactorInteger[%]
Out[9]= {{2, 26}, {3, 14}, {5, 7}, {7, 4}, {11, 2},
         {13, 2}, {17, 1}, {19, 1}, {23, 1}, {29, 1}}
```

Although *Mathematica* may not be able to factor a large integer, it can often still test whether or not the integer is a prime. In addition, *Mathematica* has a fast way to find the k^{th} prime number.

It is often much faster to test whether a number is prime than to factor it.

```
In[10]:= PrimeQ[234242423]
Out[10]= False
```

Here is a plot of the first 100 primes.



This is the millionth prime.

```
In[12]:= Prime[1000000]
Out[12]= 15485863
```

<code>PowerMod[a, b, n]</code>	the power a^b modulo n
<code>EulerPhi[n]</code>	the Euler totient function $\phi(n)$
<code>MoebiusMu[n]</code>	the Möbius function $\mu(n)$
<code>DivisorSigma[k, n]</code>	the divisor function $\sigma_k(n)$
<code>JacobiSymbol[n, m]</code>	the Jacobi symbol $\left(\frac{n}{m}\right)$
<code>ExtendedGCD[m, n]</code>	the extended gcd of m and n
<code>LatticeReduce[{v₁, v₂, ...}]</code>	the reduced lattice basis for the set of integer vectors v_i

Some functions from number theory.

The **modular power function** `PowerMod[a, b, n]` gives exactly the same results as `Mod[ab, n]`. `PowerMod` is much more efficient, however, because it avoids generating the full form of a^b .

You can use `PowerMod` not only to find positive modular powers, but also to find **modular inverses**. `PowerMod[a, -b, n]` gives, if possible, an integer k such that $(ak)^b \equiv 1 \pmod{n}$. (Whenever such an integer exists, it is guaranteed to be unique.) If no such integer k exists, *Mathematica* leaves `PowerMod` unevaluated.

`PowerMod` is equivalent to using `Power`, then `Mod`, but is much more efficient.

```
In[13]:= PowerMod[2, 13451, 3]
Out[13]= 2
```

This gives the modular inverse of 3 modulo 7.

```
In[14]:= PowerMod[3, -1, 7]
Out[14]= 5
```

Web sample page from The *Mathematica* Book, First Edition, by Stephen Wolfram, published by Addison-Wesley Publishing Company (hardcover ISBN 0-201-19334-5; softcover ISBN 0-201-19330-2). To order *Mathematica* or this book contact Wolfram Research: info@wolfram.com; <http://www.wolfram.com/>; 1-800-441-6284.

© 1988 Wolfram Research, Inc. Permission is hereby granted for web users to make one paper copy of this page for their personal use. Further reproduction, or any copying of machine-readable files (including this one) to any server computer, is strictly prohibited.

Multiplying the inverse by 3 modulo 7 gives 1, as expected.

```
In[15]:= Mod[3 %, 7]
Out[15]= 1
```

The **Euler totient function** $\phi(n)$ gives the number of integers less than n that are relatively prime to n . An important relation (Fermat's Little Theorem) is that $a^{\phi(n)} \equiv 1 \pmod n$ for all a relatively prime to n .

The **Möbius function** $\mu(n)$ is defined to be $(-1)^k$ if n is a product of k distinct primes, and 0 if n contains a squared factor (other than 1). An important relation is the Möbius inversion formula, which states that if $g(n) = \sum_{d|n} f(d)$, then $f(n) = \sum_{d|n} \mu(d)g(n/d)$, where the sums are over all positive integers d that divide n .

The **divisor function** $\sigma_k(n)$ is the sum of the k^{th} powers of the divisors of n . The function $\sigma_0(n)$ gives the total number of divisors of n , and is often denoted $d(n)$. The function $\sigma_1(n)$, equal to the sum of the divisors of n , and is often denoted $\sigma(n)$.

For prime n , $\phi(n) = n - 1$.

```
In[16]:= EulerPhi[17]
Out[16]= 16
```

The result is 1, as guaranteed by Fermat's Little Theorem.

```
In[17]:= PowerMod[3, %, 17]
Out[17]= 1
```

This gives a list of all the divisors of 24.

```
In[18]:= Divisors[24]
Out[18]= {1, 2, 3, 4, 6, 8, 12, 24}
```

$\sigma_0(n)$ gives the total number of distinct divisors of 24.

```
In[19]:= DivisorSigma[0, 24]
Out[19]= 8
```

The **Jacobi symbol** `JacobiSymbol[n, m]` reduces to the **Legendre symbol** $\left(\frac{n}{m}\right)$ when m is an odd prime. The Legendre symbol is equal to zero if n is divisible by m , otherwise it is equal to 1 if n is a quadratic residue modulo the prime m , and to -1 if it is not. An integer n relatively prime to m is said to be a quadratic residue modulo m if there exists an integer k such that $k^2 \equiv n \pmod m$. The full Jacobi symbol is a product of the Legendre symbols $\left(\frac{n}{p_i}\right)$ for each of the prime factors p_i such that $m = \prod_i p_i$.

The **extended gcd** `ExtendedGCD[m, n]` gives a list $\{g, \{r, s\}\}$ where g is the greatest common divisor of m and n , and r and s are integers such that $g = rm + sn$. The extended gcd is important in finding integer solutions to linear (Diophantine) equations.

The first number in the list is the gcd of 105 and 196.

```
In[20]:= ExtendedGCD[105, 196]
Out[20]= {7, {15, -8}}
```

The second pair of numbers satisfies $g = rm + sn$.

```
In[21]:= 15 105 - 8 196
Out[21]= 7
```

The lattice reduction function `LatticeReduce[{ $v_1$ ,  $v_2$ , ...}]` is used in many modern number theoretical and combinatorial algorithms. The basic idea is to think of the vectors v_k of integers as defining a mathematical *lattice*. The vector representing each point in the lattice can be written as a linear combination of the form $\sum c_k v_k$, where the c_k are integers. For a particular lattice, there are many possible choices of the “basis vectors” v_k . What `LatticeReduce` does is to find a reduced set of basis vectors $\bar{v}_k$ for the lattice, with certain special properties.

Three unit vectors along the three coordinate axes already form a reduced basis.

```
In[22]:= LatticeReduce[{{1,0,0},{0,1,0},{0,0,1}}]
Out[22]= {{1, 0, 0}, {0, 1, 0}, {0, 0, 1}}
```

This gives the reduced basis for a lattice in four-dimensional space specified by three vectors.

```
In[23]:= LatticeReduce[{{1,0,0,12345}, {0,1,0,12435},
                        {0,0,1,12354}}]
Out[23]= {{-1, 0, 1, 9}, {9, 1, -10, 0},
          {85, -143, 59, 6}}
```

Notice that in the last example, `LatticeReduce` replaces vectors that are nearly parallel by vectors that are more perpendicular. In the process, it finds some quite short basis vectors.